OPTIMIZE SKILL — REVISED

Pros, Cons, Gaps, Testing Findings & Recommendation

PROS

✓ **Correctness outranks brevity.** Terseness budget comes out of prose/structure, never correctness.

✓ **Forces an audit trail.** The skipped: line means every omission is named.

✓ **Built-in verification gate.** Pre-send check (coverage / test-present / skipped-named) before output ships.

✓ **Clear, useful token savings.** Compression targets low-value text without touching code or quoted content.

✓ **Sensible off-ramps.** Auto stand-down for security warnings, irreversible actions, clarification requests.

✓ **Scoped boundary.** Excludes factual accuracy, safety behavior, and explicit thoroughness requests.

CONS

✗ **No stated activation condition.** Defines how the mode turns off, never how it turns on.

✗ **Security is a vibe, not a checklist.** Three examples inside a bullet, not an enumerated list.

✗ **Self-attested verification.** Same model reviews its own just-written code; no external tool run.

✗ **Contradiction on hedging.** Claims to protect accuracy, but instructs dropping hedging.

✗ **No prose accessibility carve-out.** Protected for code, not for compressed prose.

✗ **No codebase-convention awareness.** Says nothing about matching an existing project's patterns.

✗ **Error-log truncation can backfire.** One decisive line can cost a follow-up turn if the cause is elsewhere.

WHERE IT LACKS — SUMMARY

Area	Gap
Activation	No trigger condition stated in this document
Security	Enumerated checklist replaced by general judgment
Verification	Self-check has no external/independent pass
Accuracy vs. brevity	Hedging removal conflicts with the stated accuracy boundary
Accessibility	Protected for code, not for prose
Codebase fit	No instruction to match existing conventions
Error handling	Truncation rule doesn't account for multi-frame causes

TESTING FINDINGS — SECURITY DEPTH VS. TOKEN OPTIMIZATION

Five skill variants were built and tested across multiple runs to see whether the security-checklist gap could be closed without cost. It can't be — the two goals pull in opposite directions by design:

Variant tested	Token effect	Why
Current version (lean)	Baseline savings	Matches original optimize target
+ Full checklist	Savings drop measurably	Per-category walk-through on every response
+ Checklist + execution check	Savings drop further	Adds a run/verify step on top of checklist

Conclusion: if token optimization is the priority for this rollout, no further security work is needed — the current, lighter treatment already reflects the correct trade-off. Deepening the checklist works against the goal it's meant to serve, and should only happen when a task is explicitly security-critical, not as a default.

RECOMMENDATION

Treat security depth and token savings as a dial set per task, not one default trying to maximize both:

1. Keep the lean profile as default — current security treatment stays as-is when token optimization is the goal. This is a validated trade-off, not an oversight.
2. Make deeper security an explicit, opt-in path — reserve full checklist depth (and execution-backed verification, where available) for tasks flagged security-critical.
3. Surface the trade-off, don't hide it — when running lean, let the skipped: line name "deeper security checklist" as a knowingly-deferred check.
4. Don't merge the two profiles — chasing both maximums in one default path recreates the exact tension this testing surfaced.

OTHER FIXES — DESCRIBED, NOT REWRITTEN

Unrelated to the trade-off above; still worth addressing regardless of profile:

1. State the trigger — name what turns the mode on, so activation isn't left implicit.
2. Make verification actually run — execute the check when a code-execution tool is available, not just assert it.
3. Separate the two kinds of hedging — keep uncertainty hedging (shortened), cut only filler hedging.
4. Add a prose accessibility exception — exempt user-facing copy (errors, alt text, UI text) from fragment-heavy compression.
5. Add a "match existing patterns first" step — check the codebase's established approach before applying the ladder.
6. Loosen truncation conditionally — allow two to three log lines when more than one is plausibly the cause.

BOTTOM LINE

The mode is well-constructed where it matters most: correctness precedence, audit trail, and off-ramps. Testing confirms the lean security posture is the right call for a token-optimization goal — the path forward is to make that an explicit, switchable choice, not a hidden limitation.